



AI with **Thiru**

LEARN · BUILD · DEPLOY

The 4 Types of **Fine-Tuning**

A clear, practical guide to Full Fine-Tuning, PEFT, LoRA, and QLoRA.
What each one is, when to use it, and how to implement all four in
Python.

Full Fine-Tuning

PEFT (family)

LoRA

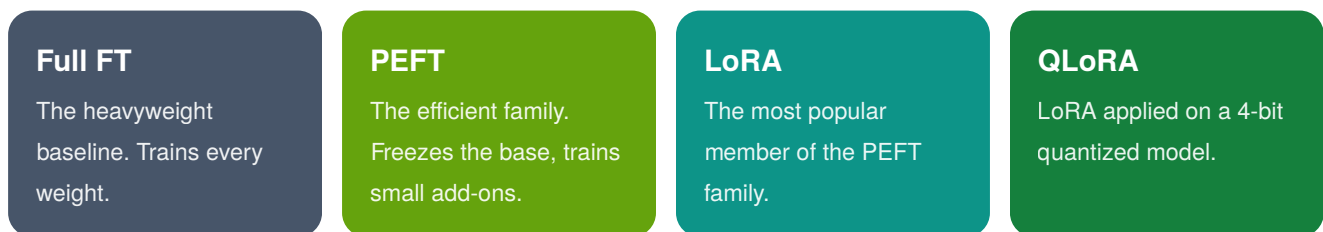
QLoRA

What is Fine-Tuning?

Fine-tuning takes a pretrained language model that already understands general language, then trains it further on your own task-specific data. The goal is to make the model behave the way you want, matching your tone, your domain, or a specific output format.

The four approaches in this guide sit on a spectrum. At one end you update every weight in the model. At the other end you update only a tiny fraction of weights on a compressed model. They are not four rival techniques. They build on one another.

How the four relate



Read the chain like this: PEFT is the family of efficient methods. LoRA is its star member. QLoRA is simply LoRA running on a quantized (compressed) model. Full Fine-Tuning stands apart as the heaviest option, updating everything.

A simple analogy

Full Fine-Tuning is like retraining an employee from scratch. The efficient methods are like handing that same employee a focused cheat sheet for one specific job. The base knowledge stays intact, and you add a small, specialized layer on top. That is why the efficient methods are faster, cheaper, and produce tiny files you can swap in and out.

OVERVIEW

The Four at a Glance

Type	Core idea	Trainable weights
Full FT	Update every parameter in the model	100%
PEFT	Freeze the base, train small add-on modules	Under 1%
LoRA	Inject trainable low-rank adapter matrices	About 0.1 to 1%
QLoRA	Run LoRA on a 4-bit quantized base model	About 0.1 to 1%

Setup: install once

Every code sample in this guide uses the Hugging Face stack. Install everything with a single command.

TERMINAL

```
# One-time install for all four methods
pip install torch transformers datasets accelerate peft bitsandbytes
```

About the examples. Each script uses a small base model, `meta-llama/Llama-3.2-1B`, and a JSON Lines dataset named `train.jsonl` where every line looks like `{"text": "your training example"}`. Swap in any causal language model and your own data.

Why the efficient methods win on storage

Because PEFT, LoRA, and QLoRA only save a small adapter, you can keep a single base model on disk and store many task-specific adapters beside it. Each adapter is just a few megabytes, so one model can serve dozens of tasks without dozens of full copies.

Rule of thumb. Full Fine-Tuning writes gigabytes per task. The efficient methods write megabytes. That difference is what makes them practical on everyday hardware.

METHOD ONE

1

Full Fine-Tuning

What it is: Every parameter of the model is unfrozen and updated during training. The model has maximum freedom to adapt, which can give the best quality. In return it needs the most GPU memory, the most data, and it produces a full-size copy of the model for every task.

Use it when you have substantial compute (large or multiple GPUs), a sizeable dataset, and you need the absolute best task performance, or you want to change the model's behavior deeply.

PYTHON FULL_FINETUNE.PY

```
from datasets import load_dataset
from transformers import (
    AutoModelForCausalLM, AutoTokenizer, TrainingArguments,
    Trainer, DataCollatorForLanguageModeling,
)

model_name = "meta-llama/Llama-3.2-1B"
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

# Load the model with ALL parameters trainable
model = AutoModelForCausalLM.from_pretrained(model_name)

# Each line of train.jsonl is {"text": "..."}
dataset = load_dataset("json", data_files="train.jsonl")["train"]
tokenized = dataset.map(
    lambda b: tokenizer(b["text"], truncation=True, max_length=512),
    batched=True, remove_columns=dataset.column_names)

args = TrainingArguments(
    output_dir="./full-ft",
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,
    num_train_epochs=3,
    learning_rate=2e-5,          # small LR: every weight moves
    fp16=True, logging_steps=10,
)

trainer = Trainer(
    model=model, args=args, train_dataset=tokenized,
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False),
)
trainer.train()
trainer.save_model("./full-ft-final")  # saves the whole model (GBs)
```

PEFT, the Family

What it is: Parameter-Efficient Fine-Tuning is an umbrella term, not a single algorithm. The shared idea is to **freeze the pretrained weights** and train only a small set of new parameters. This slashes memory use and produces tiny task files measured in megabytes. LoRA and QLoRA belong to this family, alongside Prompt Tuning and Prefix Tuning.

Use it when you want most of the benefit of fine-tuning at a fraction of the cost, and you may need many task-specific variants of one base model.

The example uses **Prompt Tuning**, a family member different from LoRA. The shared workflow: wrap the frozen base with `get_peft_model()` and train only the add-on.

PYTHON PEFT_PROMPT_TUNING.PY

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import (
    get_peft_model, PromptTuningConfig,
    PromptTuningInit, TaskType,
)

model_name = "meta-llama/Llama-3.2-1B"
tokenizer = AutoTokenizer.from_pretrained(model_name)
base_model = AutoModelForCausalLM.from_pretrained(model_name)
# Learn a few "virtual token" embeddings. The base stays frozen.
peft_config = PromptTuningConfig(
    task_type=TaskType.CAUSAL_LM,
    prompt_tuning_init=PromptTuningInit.TEXT,
    num_virtual_tokens=8,
    prompt_tuning_init_text="Classify the sentiment of this review:",
    tokenizer_name_or_path=model_name,
)

model = get_peft_model(base_model, peft_config)
model.print_trainable_parameters()
# trainable params: 16,384 || all params: 1.2B || trainable: 0.0013%

# Train with the same Trainer setup shown in Method 1,
# then save just the small prompt module:
model.save_pretrained("./peft-adapter") # a few KB
```

Key takeaway. Look at the trainable percentage: well under 1%. That single line, `get_peft_model(base, config)`, is the heart of the entire PEFT family.

LoRA

What it is: Low-Rank Adaptation is the most widely used PEFT method. Instead of editing a big weight matrix, LoRA freezes it and learns a small pair of low-rank matrices, then adds their product as an update. Only that small pair trains, so the adapter is tiny and can be swapped in and out at inference.

Use it when you want strong quality close to full fine-tuning, fast training, and small portable adapters. This is the sensible default for most tasks on a single GPU.

Two knobs that matter most. **r (rank)** sets the size of the low-rank update (common values 8 to 64: higher means more capacity). **lora_alpha** scales the update, often set to twice the rank.

PYTHON LORA_FINETUNE.PY

```
from datasets import load_dataset
from transformers import (AutoModelForCausalLM, AutoTokenizer,
                          TrainingArguments, Trainer, DataCollatorForLanguageModeling)
from peft import LoraConfig, get_peft_model, TaskType

model_name = "meta-llama/Llama-3.2-1B"
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token
model = AutoModelForCausalLM.from_pretrained(model_name)

# Define the low-rank adapters
lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16,                # rank of the update
    lora_alpha=32,        # scaling (alpha = 2 * r)
    lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
    bias="none",
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()

dataset = load_dataset("json", data_files="train.jsonl")["train"]
tokenized = dataset.map(
    lambda b: tokenizer(b["text"], truncation=True, max_length=512),
    batched=True, remove_columns=dataset.column_names)

args = TrainingArguments(output_dir="./lora-out",
                        per_device_train_batch_size=4, gradient_accumulation_steps=4,
                        num_train_epochs=3, learning_rate=2e-4, fp16=True)

Trainer(model=model, args=args, train_dataset=tokenized,
        data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False)).train()
model.save_pretrained("./lora-adapter") # only adapter, a few MB
```

QLoRA

What it is: Quantized LoRA takes LoRA one step further. The frozen base model is loaded in **4-bit precision** using bitsandbytes, which dramatically cuts memory, while the LoRA adapters still train in higher precision. This is what lets people fine-tune large models on a single consumer GPU.

Use it when the model is too big to fit in memory otherwise, or you only have one modest GPU. You get near LoRA-level quality at a fraction of the memory.

PYTHON QLORA_FINETUNE.PY

```
import torch
from datasets import load_dataset
from transformers import (AutoModelForCausalLM, AutoTokenizer,
                          BitsAndBytesConfig, TrainingArguments, Trainer,
                          DataCollatorForLanguageModeling)
from peft import (LoraConfig, get_peft_model,
                  prepare_model_for_kbit_training, TaskType)

model_name = "meta-llama/Llama-3.2-1B"

# 1) Load the base model in 4-bit
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",          # NormalFloat4
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
)
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token
model = AutoModelForCausalLM.from_pretrained(
    model_name, quantization_config=bnb_config, device_map="auto")

# 2) Prep the 4-bit model, then attach LoRA (same config as Method 3)
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, LoraConfig(
    task_type=TaskType.CAUSAL_LM, r=16, lora_alpha=32,
    lora_dropout=0.05, bias="none",
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"]))

# 3) Same data and Trainer flow as before
dataset = load_dataset("json", data_files="train.jsonl")["train"]
tokenized = dataset.map(lambda b: tokenizer(
    b["text"], truncation=True, max_length=512),
    batched=True, remove_columns=dataset.column_names)

args = TrainingArguments(output_dir="./qlora-out",
    per_device_train_batch_size=4, gradient_accumulation_steps=4,
    num_train_epochs=3, learning_rate=2e-4, bf16=True)

Trainer(model=model, args=args, train_dataset=tokenized,
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False)).train()
model.save_pretrained("./qlora-adapter")
```

Side-by-Side Comparison

Method	Trainable	Base weights	GPU memory	Output size	Best for
Full FT	100%	Updated	Very high	Full model (GBs)	Maximum quality
PEFT	Under 1%	Frozen	Low	Small adapter	Many task variants
LoRA	0.1 to 1%	Frozen	Low to medium	Adapter (few MB)	Best default
QLoRA	0.1 to 1%	Frozen (4-bit)	Very low	Adapter (few MB)	Big model, one GPU

The relationship in one line. Full FT trains everything. PEFT is the efficient family. LoRA is its star member. QLoRA is LoRA on a quantized model.

How to choose

- **Start with LoRA.** It is the pragmatic default: fast, cheap, portable, and close to full fine-tuning quality for most tasks.
- **Running out of GPU memory?** Switch to QLoRA. It is the same recipe with the base model loaded in 4-bit.
- **Need the best possible quality and have the hardware?** Use Full Fine-Tuning.
- **Want lightweight alternatives** such as prompt or prefix tuning? Reach for the broader PEFT family.

RECAP

The Four, in One View

A final summary you can return to. Each method is one sentence, in the order you would usually reach for them.

LoRA Your default starting point

Trains tiny low-rank adapters on a frozen base. Fast, cheap, portable, and close to full quality for most tasks.

QLoRA When memory is tight

The same LoRA recipe with the base model loaded in 4-bit. Lets you fine-tune large models on a single consumer GPU.

PEFT The wider toolbox

The family that LoRA and QLoRA belong to. Reach here for lightweight alternatives like prompt or prefix tuning.

Full FT When quality is critical

Trains every weight for the highest quality ceiling. Choose it when you have the data and the compute to match.



Thanks for reading

AI WITH THIRU

If this guide helped, subscribe to **AI with Thiru** for more hands-on AI, machine learning, and LLM tutorials. Try running each script on your own dataset and compare the results.